

DBMS LAB – 8: TRANSACTION ISOLATION LEVELS

Name: Adishree Gupta

SRN: PES1UG23CS024

Section: 5A

Subject: Database Management Systems Lab

Date: 17-10-2025

Objective

To learn and demonstrate the working of different **transaction isolation levels** in MySQL — **READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, and SERIALIZABLE** — and observe their effects on concurrent transactions.

Question 1: Identify the Default Isolation Level

Code

```
SELECT @@transaction_isolation;
SELECT @@global.transaction_isolation;
```

Output

```
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current
mysql> SELECT @@transaction_isolation;
+-----+
| @@transaction_isolation |
+-----+
| REPEATABLE-READ        |
+-----+
1 row in set (0.00 sec)

mysql> SELECT @@global.transaction_isolation;
+-----+
| @@global.transaction_isolation |
+-----+
| REPEATABLE-READ                |
+-----+
1 row in set (0.00 sec)

mysql> |
```

By default, MySQL uses the **REPEATABLE READ** isolation level.

Question 2: Database Setup

Code

```
CREATE DATABASE airlines_db;
USE airlines_db;

CREATE TABLE Flight (
    flight_number VARCHAR(10) PRIMARY KEY,
    departure_time DATETIME NOT NULL,
    arrival_time DATETIME NOT NULL,
    origin VARCHAR(50) NOT NULL,
    destination VARCHAR(50) NOT NULL,
    economy_seats INT NOT NULL,
    business_seats INT NOT NULL,
    first_class_seats INT NOT NULL
);

CREATE TABLE Passenger (
    passenger_id VARCHAR(10) PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    gender ENUM('Male', 'Female', 'Other'),
    date_of_birth DATE,
    phone_number VARCHAR(15),
    seat_preference ENUM('Window', 'Aisle', 'Middle'),
    class_preference ENUM('Economy', 'Business', 'First'),
    loyalty_program_points INT DEFAULT 0
);

INSERT INTO Flight VALUES
('SKY101', '2025-07-12 06:00:00', '2025-07-12 10:30:00', 'Delhi',
'Singapore', 180, 30, 10),
('SKY202', '2025-07-13 14:00:00', '2025-07-13 18:00:00', 'Mumbai', 'Dubai',
160, 25, 8),
('SKY303', '2025-07-14 22:00:00', '2025-07-15 02:30:00', 'Chennai',
'Bangkok', 150, 20, 5);

INSERT INTO Passenger VALUES
('P001', 'Aarav Sharma', 'Male', '1990-08-15', '9876543210', 'Window',
'Economy', 12000),
('P002', 'Meera Patel', 'Female', '1985-02-20', '9123456789', 'Aisle',
'Business', 22000),
('P003', 'Riya Das', 'Female', '1998-11-10', '9988776655', 'Window',
'First', 30000);
```

Output

```
mysql> USE airlines_db;
Database changed
mysql>
mysql> CREATE TABLE Flight (
->   flight_number VARCHAR(10) PRIMARY KEY,
->   departure_time DATETIME NOT NULL,
->   arrival_time DATETIME NOT NULL,
->   origin VARCHAR(50) NOT NULL,
->   destination VARCHAR(50) NOT NULL,
->   economy_seats INT NOT NULL,
->   business_seats INT NOT NULL,
->   first_class_seats INT NOT NULL
-> );
Query OK, 0 rows affected (0.13 sec)

mysql>
mysql> CREATE TABLE Passenger (
->   passenger_id VARCHAR(10) PRIMARY KEY,
->   name VARCHAR(50) NOT NULL,
->   gender ENUM('Male', 'Female', 'Other'),
->   date_of_birth DATE,
->   phone_number VARCHAR(15),
->   seat_preference ENUM('Window', 'Aisle', 'Middle'),
->   class_preference ENUM('Economy', 'Business', 'First'),
->   loyalty_program_points INT DEFAULT 0
```

```
MySQL 8.0 Command Line Cli  x  +  v
mysql>
mysql> CREATE TABLE Passenger (
->   passenger_id VARCHAR(10) PRIMARY KEY,
->   name VARCHAR(50) NOT NULL,
->   gender ENUM('Male', 'Female', 'Other'),
->   date_of_birth DATE,
->   phone_number VARCHAR(15),
->   seat_preference ENUM('Window', 'Aisle', 'Middle'),
->   class_preference ENUM('Economy', 'Business', 'First'),
->   loyalty_program_points INT DEFAULT 0
-> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> INSERT INTO Flight VALUES
-> ('SKY101', '2025-07-12 06:00:00', '2025-07-12 10:30:00', 'Delhi', 'Singapore', 180, 30, 10),
-> ('SKY202', '2025-07-13 14:00:00', '2025-07-13 18:00:00', 'Mumbai', 'Dubai', 160, 25, 8),
-> ('SKY303', '2025-07-14 22:00:00', '2025-07-15 02:30:00', 'Chennai', 'Bangkok', 150, 20, 5);
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Passenger VALUES
-> ('P001', 'Aarav Sharma', 'Male', '1990-08-15', '9876543210', 'Window', 'Economy', 12000),
-> ('P002', 'Meera Patel', 'Female', '1985-02-20', '9123456789', 'Aisle', 'Business', 22000),
-> ('P003', 'Riya Das', 'Female', '1998-11-10', '9988776655', 'Window', 'First', 30000);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
```

Question 3: READ UNCOMMITTED

Code

Session A

```
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
START TRANSACTION;
```

```
UPDATE Passenger
SET seat_preference = 'Aisle'
WHERE passenger_id = 'P001';
-- Do not commit yet
```

```
mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
Query OK, 0 rows affected (0.02 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE Passenger
      -> SET seat_preference = 'Aisle'
      -> WHERE passenger_id = 'P001';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> -- Do not commit yet
mysql> |
```

Session B

```
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
START TRANSACTION;
```

```
SELECT name, seat_preference
FROM Passenger
WHERE passenger_id = 'P001';
```

Output

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE Passenger
      -> SET seat_preference = 'Aisle'
      -> WHERE passenger_id = 'P001';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> -- Do not commit yet
mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
Query OK, 0 rows affected (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> SELECT name, seat_preference
      -> FROM Passenger
      -> WHERE passenger_id = 'P001';
+-----+-----+
| name      | seat_preference |
+-----+-----+
| Aarav Sharma | Aisle          |
+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Even though Session A has **not committed**, Session B can see the updated value.
This is a **Dirty Read**.

Question 4: READ COMMITTED

Code

Session A

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
```

```
SELECT seat_preference
FROM Passenger
WHERE passenger_id = 'P001';
```

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql>
mysql> SELECT seat_preference
        -> FROM Passenger
        -> WHERE passenger_id = 'P001';
```

```
+-----+
| seat_preference |
+-----+
| Aisle          |
+-----+
1 row in set (0.00 sec)
```

```
mysql>
```

Session B

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
```

```
UPDATE Passenger
SET seat_preference = 'Window'
WHERE passenger_id = 'P001';
COMMIT;
```

```
mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql>
mysql> UPDATE Passenger
        -> SET seat_preference = 'Window'
        -> WHERE passenger_id = 'P001';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

```
mysql>
```

Session A (again)

```
SELECT seat_preference
FROM Passenger
WHERE passenger_id = 'P001';
```

Output

```
mysql> SELECT seat_preference
-> FROM Passenger
-> WHERE passenger_id = 'P001';
+-----+
| seat_preference |
+-----+
| Window          |
+-----+
1 row in set (0.00 sec)

mysql> ^N
```

The value read by Session A changes within the same transaction, showing a **Non-Repeatable Read**.

Question 5: REPEATABLE READ

Code

Session A

```
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;
```

```
SELECT passenger_id, name, seat_preference
FROM Passenger
WHERE class_preference = 'Economy';
```

```
mysql> SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
Query OK, 0 rows affected (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT passenger_id, name, seat_preference
-> FROM Passenger
-> WHERE class_preference = 'Economy';
+-----+-----+-----+
| passenger_id | name       | seat_preference |
+-----+-----+-----+
| P001         | Aarav Sharma | Window          |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Session B

```
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;
```

```
UPDATE Passenger
SET seat_preference = 'Aisle'
WHERE passenger_id = 'P001';
COMMIT;
```

```
mysql> SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
Query OK, 0 rows affected (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE Passenger
      -> SET seat_preference = 'Aisle'
      -> WHERE passenger_id = 'P001';
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

Session A (again)

```
SELECT passenger_id, name, seat_preference
FROM Passenger
WHERE class_preference = 'Economy';
```

Output

```
mysql> SELECT passenger_id, name, seat_preference
      -> FROM Passenger
      -> WHERE class_preference = 'Economy';
+-----+-----+-----+
| passenger_id | name       | seat_preference |
+-----+-----+-----+
| P001         | Aarav Sharma | Aisle          |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Session A still sees the **old value**, proving **Repeatable Read** consistency.

Question 6: SERIALIZABLE

Code

Session A

```
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION;
```

```
SELECT COUNT(*)
FROM Flight
WHERE DATE(departure_time) = '2025-07-14'
FOR UPDATE;
```

```
mysql> SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.01 sec)
```

```
mysql>
mysql> SELECT COUNT(*)
      -> FROM Flight
      -> WHERE DATE(departure_time) = '2025-07-14'
      -> FOR UPDATE;
```

```
+-----+
| COUNT(*) |
+-----+
|         1 |
+-----+
1 row in set (0.01 sec)
```

```
mysql> |
```

Session B

```
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION;
```

```
INSERT INTO Flight
VALUES ('SKY404', '2025-07-14 08:00:00', '2025-07-14 12:30:00', 'Delhi',
'Colombo', 170, 25, 6);
```

Output

```
mysql>
mysql> SELECT COUNT(*)
      -> FROM Flight
      -> WHERE DATE(departure_time) = '2025-07-14'
      -> FOR UPDATE;
+-----+
| COUNT(*) |
+-----+
|         1 |
+-----+
1 row in set (0.01 sec)

mysql> SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
Query OK, 0 rows affected (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO Flight
      -> VALUES ('SKY404', '2025-07-14 08:00:00', '2025-07-14 12:30:00', 'Delhi', 'Colombo', 170, 25, 6);
Query OK, 1 row affected (0.01 sec)

mysql>
```

Session B is blocked until Session A commits — showing full **serialization** and prevention of concurrent updates.

Question 7: Observation Table

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Comment
Read Uncommitted	Allowed	Possible	Possible	Lowest consistency
Read Committed	Prevented	Possible	Possible	Default in Oracle
Repeatable Read	Prevented	Prevented	Possible	MySQL default
Serializable	Prevented	Prevented	Prevented	Highest consistency, slowest concurrency

Question 8: Conclusion

In this experiment, we observed how **isolation levels** affect data visibility across concurrent transactions.

As we move from **Read Uncommitted** → **Serializable**, data consistency improves but concurrency decreases.

MySQL's default level (**Repeatable Read**) balances performance and consistency for most applications.

Time taken for SERIALIZABLE Insertion into Flight (in Secs)

```
-- =====  
-- MEASURING TIME FOR SERIALIZABLE INSERTION  
-- =====  
  
-- SESSION A (Run in first MySQL terminal/tab)  
-- =====  
  
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
START TRANSACTION;  
  
-- Record start time  
SELECT NOW(6) AS 'Session_A_Start_Time';  
  
SELECT COUNT(*)  
FROM Flight  
WHERE DATE(departure_time) = '2025-07-14'  
FOR UPDATE;  
  
-- WAIT HERE - Don't commit yet!  
-- Let Session B attempt its INSERT first  
  
  
-- SESSION B (Run in second MySQL terminal/tab)  
-- =====  
  
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
START TRANSACTION;
```

```
-- Record the time BEFORE attempting INSERT
```

```
SELECT NOW(6) AS 'Session_B_INSERT_Attempt_Time';
```

```
-- This will BLOCK until Session A commits
```

```
INSERT INTO Flight
```

```
VALUES ('SKY404', '2025-07-14 08:00:00', '2025-07-14 12:30:00', 'Delhi',
```

```
'Colombo', 170, 25, 6);
```

```
-- This will execute AFTER Session A commits
```

```
SELECT NOW(6) AS 'Session_B_INSERT_Complete_Time';
```

```
COMMIT;
```

```
-- SESSION A (Continue - Run after Session B is blocked)
```

```
-- =====
```

```
-- Wait a few seconds (e.g., 5-10 seconds) to demonstrate blocking
```

```
-- Then commit to release the lock
```

```
SELECT NOW(6) AS 'Session_A_Commit_Time';
```

```
COMMIT;
```

```
-- =====
```

```
-- CALCULATING THE BLOCKING TIME
```

```
-- =====
```

-- After both sessions complete, run this to see all times:

-- The blocking time = Session_A_Commit_Time - Session_B_INSERT_Attempt_Time

-- This shows how long Session B waited for the lock

-- Example calculation:

-- If Session B attempted INSERT at: 10:30:45.123456

-- And Session A committed at: 10:30:50.234567

-- Blocking time = 5.111111 seconds

-- =====

-- ALTERNATIVE: Using a Timing Table

-- =====

-- Create a table to store timing results

CREATE TABLE IF NOT EXISTS timing_results (

session VARCHAR(10),

event VARCHAR(50),

event_time DATETIME(6)

);

-- SESSION A with timing table:

SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;

START TRANSACTION;

INSERT INTO timing_results VALUES ('A', 'Start', NOW(6));

```
SELECT COUNT(*)  
FROM Flight  
WHERE DATE(departure_time) = '2025-07-14'  
FOR UPDATE;
```

```
-- WAIT for Session B to attempt INSERT
```

```
-- Then after waiting (e.g., 5 seconds):
```

```
INSERT INTO timing_results VALUES ('A', 'Commit', NOW(6));  
COMMIT;
```

```
-- SESSION B with timing table:
```

```
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
START TRANSACTION;
```

```
INSERT INTO timing_results VALUES ('B', 'INSERT_Attempt', NOW(6));
```

```
INSERT INTO Flight  
VALUES ('SKY404', '2025-07-14 08:00:00', '2025-07-14 12:30:00', 'Delhi',  
'Colombo', 170, 25, 6);
```

```
INSERT INTO timing_results VALUES ('B', 'INSERT_Complete', NOW(6));  
COMMIT;
```

```
-- View all timing results:
```

```
SELECT * FROM timing_results ORDER BY event_time;
```

-- Calculate blocking duration:

```
SELECT
```

```
    TIMESTAMPDIFF(MICROSECOND,
```

```
        (SELECT event_time FROM timing_results WHERE session = 'B' AND event =  
'INSERT_Attempt'),
```

```
        (SELECT event_time FROM timing_results WHERE session = 'B' AND event =  
'INSERT_Complete')
```

```
    ) / 1000000.0 AS 'Blocking_Time_Seconds';
```

Output

```
mysql> -- =====  
mysql> -- MEASURING TIME FOR SERIALIZABLE INSERTION  
mysql> -- =====  
mysql>  
mysql> -- SESSION A (Run in first MySQL terminal/tab)  
mysql> -- =====  
mysql>  
mysql> SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
Query OK, 0 rows affected (0.06 sec)  
  
mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.03 sec)  
  
mysql>  
mysql> -- Record start time  
mysql> SELECT NOW(6) AS 'Session_A_Start_Time';  
+-----+  
| Session_A_Start_Time |  
+-----+  
| 2025-10-17 16:02:46.099419 |  
+-----+  
1 row in set (0.01 sec)  
  
mysql>  
mysql> SELECT COUNT(*)  
-> FROM Flight  
-> WHERE DATE(departure_time) = '2025-07-14'  
-> FOR UPDATE;  
+-----+
```

```
MySQL 8.0 Command Line Cli x + v
-> FOR UPDATE;
+-----+
| COUNT(*) |
+-----+
|      2   |
+-----+
1 row in set (0.02 sec)

mysql>
mysql> -- WAIT HERE - Don't commit yet!
mysql> -- Let Session B attempt its INSERT first
mysql>
mysql>
mysql> -- SESSION B (Run in second MySQL terminal/tab)
mysql> -- =====
mysql>
mysql> SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
Query OK, 0 rows affected (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- Record the time BEFORE attempting INSERT
mysql> SELECT NOW(6) AS 'Session_B_INSERT_Attempt_Time';
+-----+
| Session_B_INSERT_Attempt_Time |
+-----+
| 2025-10-17 16:02:46.141393    |
+-----+
```

```
MySQL 8.0 Command Line Cli x + v

mysql>
mysql> -- This will BLOCK until Session A commits
mysql> INSERT INTO Flight
-> VALUES ('SKY404', '2025-07-14 08:00:00', '2025-07-14 12:30:00', 'Delhi',
-> 'Colombo', 170, 25, 6);
ERROR 1062 (23000): Duplicate entry 'SKY404' for key 'flight.PRIMARY'
mysql>
mysql> -- This will execute AFTER Session A commits
mysql> SELECT NOW(6) AS 'Session_B_INSERT_Complete_Time';
+-----+
| Session_B_INSERT_Complete_Time |
+-----+
| 2025-10-17 16:02:46.153675    |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql>
mysql> -- SESSION A (Continue - Run after Session B is blocked)
mysql> -- =====
mysql>
mysql> -- Wait a few seconds (e.g., 5-10 seconds) to demonstrate blocking
mysql> -- Then commit to release the lock
mysql>
mysql> SELECT NOW(6) AS 'Session_A_Commit_Time';
```

```
MySQL 8.0 Command Line Cli  x + v
mysql>
mysql> SELECT NOW(6) AS 'Session_A_Commit_Time';
+-----+
| Session_A_Commit_Time |
+-----+
| 2025-10-17 16:02:46.161044 |
+-----+
1 row in set (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql>
mysql> -- =====
mysql> -- CALCULATING THE BLOCKING TIME
mysql> -- =====
mysql> -- After both sessions complete, run this to see all times:
mysql> -- The blocking time = Session_A_Commit_Time - Session_B_INSERT_Attempt_Time
mysql> -- This shows how long Session B waited for the lock
mysql> -- Example calculation:
mysql> -- If Session B attempted INSERT at: 10:30:45.123456
mysql> -- And Session A committed at:      10:30:50.234567
mysql> -- Blocking time = 5.111111 seconds
mysql>
mysql>
mysql> -- =====
```

```
MySQL 8.0 Command Line Cli  x + v
mysql> -- =====
mysql> -- ALTERNATIVE: Using a Timing Table
mysql> -- =====
mysql>
mysql> -- Create a table to store timing results
mysql> CREATE TABLE IF NOT EXISTS timing_results (
  ->     session VARCHAR(10),
  ->     event VARCHAR(50),
  ->     event_time DATETIME(6)
  -> );
Query OK, 0 rows affected (0.13 sec)

mysql>
mysql> -- SESSION A with timing table:
mysql> SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
Query OK, 0 rows affected (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO timing_results VALUES ('A', 'Start', NOW(6));
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> SELECT COUNT(*)
  -> FROM Flight
  -> WHERE DATE(departure_time) = '2025-07-14'
  -> FOR UPDATE;
+-----+
```

```
MySQL 8.0 Command Line Cli  x  +  v

mysql>
mysql> -- WAIT for Session B to attempt INSERT
mysql>
mysql> -- Then after waiting (e.g., 5 seconds):
mysql> INSERT INTO timing_results VALUES ('A', 'Commit', NOW(6));
Query OK, 1 row affected (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql>
mysql> -- SESSION B with timing table:
mysql> SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
Query OK, 0 rows affected (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO timing_results VALUES ('B', 'INSERT_Attempt', NOW(6));
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO Flight
-> VALUES ('SKY404', '2025-07-14 08:00:00', '2025-07-14 12:30:00', 'Delhi',
-> 'Colombo', 170, 25, 6);
ERROR 1062 (23000): Duplicate entry 'SKY404' for key 'flight.PRIMARY'
mysql>
```

```
MySQL 8.0 Command Line Cli  x  +  v

mysql>
mysql> INSERT INTO timing_results VALUES ('B', 'INSERT_Complete', NOW(6));
Query OK, 1 row affected (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql>
mysql> -- View all timing results:
mysql> SELECT * FROM timing_results ORDER BY event_time;
+-----+-----+-----+
| session | event      | event_time                |
+-----+-----+-----+
| A       | Start      | 2025-10-17 16:02:46.300000 |
| A       | Commit     | 2025-10-17 16:02:46.311382 |
| B       | INSERT_Attempt | 2025-10-17 16:02:46.322764 |
| B       | INSERT_Complete | 2025-10-17 16:02:46.326617 |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> -- Calculate blocking duration:
mysql> SELECT
->   TIMESTAMPDIFF(MICROSECOND,
->   (SELECT event_time FROM timing_results WHERE session = 'B' AND event = 'INSERT_Attempt'),
->   (SELECT event_time FROM timing_results WHERE session = 'B' AND event = 'INSERT_Complete'))
->   / 1000000.0 AS 'Blocking_Time_Seconds';
+-----+
| Blocking_Time_Seconds |
+-----+

```

```
MySQL 8.0 Command Line C... + v
mysql>
mysql>
mysql> -- View all timing results:
mysql> SELECT * FROM timing_results ORDER BY event_time;
+-----+-----+-----+
| session | event      | event_time |
+-----+-----+-----+
| A       | Start      | 2025-10-17 16:02:46.300000 |
| A       | Commit     | 2025-10-17 16:02:46.311382 |
| B       | INSERT_Attempt | 2025-10-17 16:02:46.322764 |
| B       | INSERT_Complete | 2025-10-17 16:02:46.326617 |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> -- Calculate blocking duration:
mysql> SELECT
->     TIMESTAMPDIFF(MICROSECOND,
->     (SELECT event_time FROM timing_results WHERE session = 'B' AND event = 'INSERT_Attempt'),
->     (SELECT event_time FROM timing_results WHERE session = 'B' AND event = 'INSERT_Complete')
->     ) / 1000000.0 AS 'Blocking_Time_Seconds';
+-----+
| Blocking_Time_Seconds |
+-----+
| 0.0039 |
+-----+
1 row in set (0.01 sec)

mysql> |
```